ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PAPER 2 · PRACTICE D

MARKS: 100 **TIME: 3 hours**

NC(V) Programming and Robotics, Level 4. Python practical in the style of the DHET second papers (100 marks): a Part class for a robot spares store with an ElectronicPart subclass, a CSV job log for a robot workshop with totals per robot, and a menu-driven scoreboard for a robot competition with input validation and a sorted leaderboard.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of THREE questions. Answer ALL the questions.
2. This is a practical paper. Use a Python 3 IDE (such as Thonny or IDLE). If you do not have a computer, write the programs neatly in your answer book.
3. Save each program with the question number and your name or examination number, and add your name or examination number as a comment on the first line.
4. Print each program with line numbers. Save your work regularly. Programs are not marked strictly on syntax.

QUESTION 1

> A college robotics workshop keeps a store of spare parts. Every part has a name, a part code, a quantity in stock and a price per item. Electronic parts also have a working voltage, which must not be higher than the voltage of the robot they are fitted to.

1.1 Create a new file called Question1_YourName.py with your name or examination number as a comment on the first line. (1)

1.2 Create a class named Part with a constructor that takes name, code, price and quantity (default 0) and stores them as attributes. (6)

1.3 Add a method add_stock(n) that adds n items to the quantity. If n is not more than 0, display "Invalid quantity" instead. (4)

1.4 Add a method remove_stock(n). If there are not enough items, display "Not enough stock" and do not change the quantity; otherwise reduce the quantity and display the new quantity. (6)

1.5 Add a method value() that returns the total value of the part in stock (quantity × price). (3)

1.6 Create a class ElectronicPart that inherits from Part. Its constructor must call the parent constructor and add an attribute voltage. Add a method is_safe(robot_voltage) that returns True if the part's voltage is not higher than robot_voltage, and False otherwise. (7)

1.7 Test your classes: create an ElectronicPart named "Ultrasonic sensor" with code "US01", price 45.50, quantity 10 and voltage 5. Remove 3 items, display the value of the stock, and display whether it is safe for a 12 V robot. (3)

[30]

QUESTION 2

> Each job done by a robot in the workshop is written to a text file called jobs.csv as one line with the robot's name, the job and the minutes it took, separated by commas, for example: R2,welding,35

2.1 Create a new file called Question2_YourName.py. (1)

2.2 Write a function log_job(robot, job, minutes) that adds one job as a new line at the end of jobs.csv, without deleting the jobs already in the file. (5)

2.3 Write code that keeps asking the user for a robot name, a job and the minutes, and saves each job with log_job(), until the user types done as the robot name. Display "Minutes must be a number" and ask again if the minutes are not a whole number. (6)

2.4 Write a function totals() that reads jobs.csv and returns a dictionary with the total minutes worked by each robot. If the file does not exist, display "No jobs logged yet" and return an empty dictionary. (9)

2.5 Use totals() to display each robot with its total minutes, and then the name of the robot that worked the most minutes. (6)

2.6 Save and print the program, and a screenshot of the output. (3)

[30]

QUESTION 3

At a robot competition, every team's robot does several runs and each run gets a score out of 100. The organisers want a menu-driven program to keep the scores and show the leaderboard.

The following steps show how the program should work:

```
1 Add team  2 Add score  3 Leaderboard  4 Team average  5 Quit
Choice: 2
Team: Bravo
Score (0-100): 120
Score must be from 0 to 100.
Score (0-100): 88
Score saved.
Choice: 3
1. Charlie - 215 points (3 runs)
2. Bravo - 178 points (2 runs)
3. Alpha - 163 points (2 runs)
Choice: 1
Team name: Alpha
Alpha is already registered.
Choice: 4
Team: Delta
Delta is not registered.
Choice: 5
Goodbye!
```

3.1 Create a new file called Question3_YourName.py. (1)

3.2 Create a dictionary named scores in which each key is a team name and each value is a list of that team's scores: Alpha [78, 85], Bravo [90] and Charlie [65, 70, 80]. (3)

3.3 Write a function add_team() that asks for a team name and adds it with an empty list, or displays a message if the team is already registered. (4)

3.4 Write a function add_score() that asks for a team name (display a message if it is not registered) and then keeps asking for a score until it is a whole number from 0 to 100 (use try … except to handle text), and adds the score to that team's list. (9)

3.5	Write a function leaderboard() that displays the teams numbered from the highest to the lowest total score, with each team's total and number of runs.	(7)
3.6	Write a function team_average() that asks for a team name and displays its average score rounded to one decimal place, or a message if the team is not registered or has no scores yet.	(5)
3.7	Write the menu loop that calls the correct function for choices 1 to 4 and ends the program with "Goodbye!" for choice 5.	(7)
3.8	Display "Invalid choice." for any other input.	(2)
3.9	Test the program with the data in the example above and take a screenshot of the output.	(1)
3.10	Save and print the program with line numbers.	(1)

[40]

TOTAL: 100

END OF QUESTION PAPER